

Separating Interpretation from Authority: Bounded Local Adaptation for Desktop Automation by Demonstration

Anonymous author

1 Objective and significance

Everyday computer workflows often repeat while a few details change: a report uses a new month, an invoice uses a new amount, or a spreadsheet update targets a new cell. Recorded macros reproduce a route but are difficult for non-programmers to adapt. General computer-use agents can plan a new route, but that flexibility creates an authority problem: a user may not know which actions the model considers itself permitted to add, remove, or redirect. This paper asks whether a local multimodal model can interpret a demonstration and a later variation while deterministic software constrains the executable result.

We present Nelo, an open-source macOS programming-by-demonstration system. The contribution is not the combination of speech and demonstration, which is established by SUGILITE and PUMICE [Li et al., 2020, 2019]. Instead, Nelo explores a bounded middle ground between rigid replay and open-ended GUI agency. It targets workflows whose route remains stable while values or a small set of structured targets vary.

2 System and approach

During teaching, Nelo records a selected display, optional system audio, application and input events, and microphone narration. All evidence uses a shared session clock. The application selects event-adjacent and visually changed frames, performs local optical character recognition, maps display coordinates to image coordinates, and aligns nearby narration with each action. A quantized Qwen3-VL 4B model runs locally through MLX. It labels captured steps and can reconstruct narrowly grounded missing inputs when macOS event capture is incomplete.

The design separates interpretation from authority in two stages. First, teaching-time model proposals are marked as visual drafts, filtered against supplied evidence, and shown for review. Saving the workflow establishes the action graph. Second, a later instruction such as “Use August and \$3,000” exposes only declared variable steps to the model. Model output contains value replacements keyed by existing step identifiers—not clicks, URLs, or actions. A deterministic validator rejects unknown or repeated identifiers, non-variable steps, structural references, and unsafe text. The run planner copies the saved graph, applies accepted values, previews exact changes, checks readiness, and preserves approval gates. A request to “submit without asking” therefore cannot remove the pause. Supported structural variation, such as moving a demonstrated Google Sheets value to a validated new cell, uses an explicitly implemented “Go to range” transformation rather than a model-invented coordinate.

3 Evaluation and findings

We froze a synthetic, account-free benchmark before collection. It contains 15 cases, 61 weighted executable assertions, and 59 critical assertions. Five deterministic cases test trace compilation, fixture grounding, unchanged replay, a safe spreadsheet transformation, and known-data cross-application transfer. Ten Qwen-dependent cases test complete-capture labeling, partial and video-only recovery, single- and multi-value adaptation, recurring reports, forms, rejection of unsupported or malicious variation instructions, and preservation of an invoice-submit approval.

We ran three fresh-process trials for both the 4-bit and 8-bit model revisions with identical prompts (temperature 0.15; top-p 0.9). Reports retain failed trials rather than retrying them away. Collection used a 14-core Apple M4

Pro Mac with 48 GB memory and macOS 26.6.2. We record aggregate and model-only case pass rates, every critical failure, cached model-setup time, total evaluation time, peak resident memory, and pinned model size. All six trials passed: both tiers achieved 100.0% model-case pass rates with 0 critical failures. Peak RSS was 3.41 GiB (4-bit) and 5.28 GiB (8-bit).

The benchmark uses exact expected outputs, not a model judge. A case passes only when its weighted score is at least 0.80 and no critical assertion fails; a trial also requires every case to pass. Because the cases are designed constructs rather than a random task sample, we report descriptive results and do not generalize percentages to arbitrary desktop work.

4 Contributions and implications

The paper contributes (1) a shared-clock, local multimodal desktop teaching pipeline; (2) an explicit authority model that permits reviewed reconstruction during teaching but prohibits free-form action generation during adaptation; and (3) a reproducible evaluation package with pinned models, raw reports, analysis code, and resource measurements.

The result is formative evidence about implemented invariants, not a usability or safety certification. No human participants, personal recordings, or live third-party accounts are included. The study does not measure non-programmer effectiveness, indirect prompt injection from webpage content, layout drift, or 16 GB performance. A next phase will expand to resettable live tasks and modality ablations, followed by an ethics-reviewed study of whether non-programmers can understand and repair the boundary. Neloa suggests that useful agentic flexibility need not require granting a model continuous, open-ended control of the desktop—and that local inference can keep ordinary workflow recordings away from external AI providers while remaining honest about local privacy limits.

References

- Toby Jia-Jun Li, Marissa Radensky, Justin Jia, Kirielle Singarajah, Tom M. Mitchell, and Brad A. Myers. PUMICE: A multi-modal agent that learns concepts and conditionals from natural language and demonstrations. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*, pages 577–589. Association for Computing Machinery, 2019. doi: 10.1145/3332165.3347899.
- Toby Jia-Jun Li, Tom Mitchell, and Brad A. Myers. Interactive task learning from GUI-grounded natural language instructions and demonstrations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 215–223. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-demos.25. URL <https://aclanthology.org/2020.acl-demos.25/>.